

SYSTEM ADMINISTRATION MANUAL: ACADEMIC HELPER PLATFORM

Classification: Restricted (Administrative Reference Data Only)

Version: 2026.1

1. System Overview & Core Architecture

The Academic Helper Platform is a Retrieval-Augmented Generation (RAG) system built to serve university students and staff. It parses official institutional documentation, creates geometric vector representation fragments, and references those contexts securely through a multi-tenant cloud language model (Groq Cloud API).

The platform utilizes a modern detached-processing pipeline:

1. **Relational Layer (SQLite / SQLAlchemy):** Manages user authentications, access control flags, approval statuses, and structural chat histories.
2. **Vector Space Layer (ChromaDB):** Houses document chunks processed locally via advanced semantic mapping models, completely distinct from the relational data framework.
3. **Synthesis Engine Layer (Groq Cloud API / LPU Architecture):** Processes curated context structures using the `llama-3.3-70b-versatile` cloud infrastructure to construct deterministic responses at high speeds.

2. Ingesting & Indexing Institutional Documentation

New institutional policies, student handbooks, and regulatory frameworks must be actively processed by an Administrator to become accessible inside the Chat Workspace.

Ingestion Walkthrough:

1. Authenticate into the platform using an account assigned with `ADMIN` clearance tier privileges.
2. Navigate to the top navigation dashboard toolbar and select **Admin Queue** to open the operational Control Panel.
3. Locate the module section titled "**Upload & Index Regulation Documentation**".
4. Click the file selector interface and locate your target document. **Note:** The document must be saved natively as a standard `.pdf` configuration. Scanned image documents or raw text inputs are rejected at the interface validation layer.
5. Select the appropriate **Document Access Tier** (see Section 3 for definitions) via the clearance configuration dropdown.
6. Click **Upload & Parse to Vector Database**.

Internal Vector Processing Lifecycle:

Once submitted, the system triggers the custom processing code:

- **Extraction:** The local execution framework utilizes `pypdf` to isolate structural character sequences page-by-page.
- **Granular Chunking:** Text layers are split cleanly into paragraph slices at every double line break (`\n\n`). Slices containing fewer than 30 characters are skipped automatically to eliminate document noise (e.g., page numbers or running headers).
- **Local Embedding:** The text fragments are fed into ChromaDB's local `DefaultEmbeddingFunction`, which uses the `all-MiniLM-L6-v2` transformer model locally on the server.
- **Vector Hypersphere Insertion:** The text segments are transformed into 384-dimensional mathematical arrays and written onto disk inside the permanent `chroma_db/` folder. This process runs locally on your machine and does not transmit raw document bodies over the public internet to third-party providers.

3. Dynamic Multi-Tenant Access Control Levels

To protect institutional confidentiality, the system strictly isolates document access using automatic role-based clearance evaluations. When a question is submitted, ChromaDB applies metadata constraints (`where={"clearance": {"$in": allowed_clearances}}`) during the vector lookup phase.

The system enforces three hard security boundaries:

Clearance Level Name	Target Field Value	Authorized System Roles	Description / Operational Bounds
Public	public	Students, Faculty, Administrators	General university materials. Includes student handbooks, basic campus maps, and public course enrollment frameworks.
Internal	internal	Faculty, Administrators	Non-public institutional workflows. Includes grading schema frameworks, mitigation panel guidelines, and staff-facing administrative manuals. Standard

Clearance Level Name	Target Field Value	Authorized System Roles	Description / Operational Bounds
			student roles are fully blind to this content.
Restricted	restricted	Administrators	High-tier policy material. Restricted to senior administrative strategies, system configurations, and restricted escalation procedures.

4. Administrative Account Provisioning & Verification Workflow

To guarantee overall data integrity, the platform blocks external access using an active confirmation pipeline.

The Self-Registration Cycle:

- When a user visits the `/register` gateway, they provide their Username, Email, Password, and select their claimed target role (`Student` or `Faculty`).
- **Automated Processing Rules:**
 - **Students:** Accounts claiming the `Student` role bypass administrative confirmation. Their `is_approved` status property maps to `True` instantly upon registration, allowing immediate access to the Chat Workspace.
 - **Faculty / Staff:** Accounts claiming the `Faculty` role are initialized with `is_approved = False`. They are immediately redirected to the login page with an alert indicating that their credentials require manual system confirmation.

Approving Pending Personnel:

1. Log into your Master Admin Account.
2. Open the **Admin Queue** screen.
3. Locate the table framework labeled "**Pending Verification Items**". This area uses a live relational loop that searches your SQLite database specifically for staff rows where `is_approved == False`.

4. Review the applicant's self-reported profile data (Username, Email, and Student/Faculty ID).
5. To authorize the account, click **[GRANT APPROVAL]**. This changes the user's `is_approved` value to `True` inside the database, allowing them to log in securely and query both `public` and `internal` document clearings.

5. System Maintenance & Troubleshooting Real-Time Error Codes

When managing the live application lifecycle, administrators may occasionally encounter explicit technical notifications displayed directly within the layout interfaces.

1. Unique Constraint Failure (`sqlite3.IntegrityError`)

- **Visual Symptom:** The application triggers a 500 Server Exception during a user signup transaction, referencing a failed unique constraint on `student.student_id`.
- **Root Cause:** A user is submitting a registration form with a Student/Faculty ID string that is already assigned to an existing database record (or matches the built-in Master Administrator profile ID: `SYS-ADMIN-01`).
- **Correction Procedure:** Ensure the unique form validation mechanism `validate_student_id` is properly enabled in `app/forms.py`. Instruct the registering user to utilize their exact, unique institutional identity key rather than generic or pre-existing place markers.

2. Missing Sources / Live UI Refresh Latency

- **Visual Symptom:** An administrator asks a question in the chat workspace. The response displays the correct answer text, but the citation text fields display as blank or empty. When the webpage is refreshed manually, the correct sources and page allocations appear as expected.
- **Root Cause:** The browser's client-side asynchronous script (`index.html`) is appending only the raw text response property (`data.answer`) received from the AJAX payload while omitting the explicit dynamic token identifier (`data.citation`).
- **Correction Procedure:** Locate the `fetch` statement sequence inside `app/templates/index.html` and verify that the HTML injection string explicitly prints both attributes:

JavaScript

```
box.innerHTML += `

<strong>Bot:</strong> ${data.answer} (Source:
${data.citation})</p>`;


```

3. Cloud Configuration Faults (`GROQ_API_KEY` missing)

- **Visual Symptom:** The Chatbot outputs an explicit system error message:
"Configuration Error: GROQ_API_KEY missing from system .env file."
- **Root Cause:** The backend server environment is unable to locate the necessary cloud access token keys inside your operating system's active context workspace.
- **Correction Procedure:** Check the root project folder on the host machine. Ensure a secure text file named exactly `.env` exists. Verify that the file holds your active cloud token string using the precise declaration syntax:

Plaintext

```
GROQ_API_KEY=gsk_your_private_api_key_here
```

Restart the web container application instance (`python setup.py`) to re-initialize your system environment variables.